ARCHITECTURE V1.3 — IMPLEMENTATION BASELINE

BOS Light

Paperclip Native Adapters:
hermes_local + gsdpi_local

Версия 1.3 — исправление после adapter-инсайта
Май 2026

Содержание

1. Исправление v1.2: что было не так

Версия 1.2 (и критическая оценка отчета второго агента) содержали архитектурную ошибку: предполагалось, что Hermes и GSD-Pi подключаются как **внешние executors** через custom executor registry в BOS Light plugin worker.

Ошибка v1.2: plugin worker spawn external processes

```
BOS Light plugin worker
  └ custom executor registry
       ├ spawn Hermes (Python subprocess)
       └ spawn GSD-Pi (Node.js subprocess)
```

Это создавало третий runtime layer поверх Paperclip, дублируя то, что Paperclip уже умеет: agent lifecycle, heartbeats, sessions, logs, costs.

Эта схема ошибочна, потому что **упускает ключевой факт**: Paperclip уже имеет native adapter layer для CLI, HTTP и custom agents.

2. Ключевой инсайт: Paperclip adapter layer

Paperclip docs описывают adapters как bridge между control plane и agent runtime. Adapter определяет: как агент вызывается, как получает work context, как отчитывается обратно. Paperclip уже включает:

- **Process Adapters** — для локальных CLI tools (spawn, stdout/stderr, timeout, SIGTERM/SIGKILL, parse result)
- **HTTP Adapters** — для webhook/remote agents
- **Custom Adapters** — для произвольной среды (ServerAdapterModule: testEnvironment, execute, sessionCodec)

Paperclip adapter interface включает: `execute(ctx)`, `testEnvironment(ctx)`, `sessionCodec`, structured `AdapterExecutionResult` с `resultJson` для custom metadata.

3. Исправленная архитектура

Правильная формула

```
Paperclip Company
  ├─ Org Chart & Agents
  ├─ Issues / Tasks / Heartbeats
  ├─ Adapter Registry
  │     ├─ claude_local
  │     ├─ codex_local
  │     ├─ hermes_local        ← exists (NousResearch)
  │     ├─ gsdpi_local         ← to build (custom adapter)
  │     └─ http
  ├─ Budget / Cost / Governance
  └─ Activity / Audit trail

BOS Light Plugin
  ├─ division metadata
  ├─ BPI / Betting Table / Blueprints
  ├─ Eval Gates / Circuit Breaker
  ├─ routing / assignment policy
  └─ result governance (parses resultJson.bos)
```

Формула не изменилась: **BOS decides/annotates, Paperclip owns state, agents execute, Paperclip records/approves**. Но реализация агентов — через Paperclip native adapters, не через plugin worker spawn.

4. Hermes через hermes_local

4.1 Существующий adapter

У NousResearch уже есть `hermes-paperclip-adapter`, который регистрирует `hermes_local` в Paperclip adapter registry. Этот adapter покрывает:

- Toolsets, session codec, structured transcript parsing
- Comment-driven wakes, session persistence
- Skills integration, worktree/checkpoints
- Передачу результатов обратно в Paperclip через resultJson

Hermes adapter config поддерживает: `toolsets`, `persistSession`, `worktreeMode`, `checkpoints`, `extraArgs`, `env`, `promptTemplate`, `paperclipApiUrl`. Роль и изоляция задаются на уровне Paperclip agent config, не отдельным BOS executor manager.

4.2 6 agents с одним adapter

Не нужно 6 Hermes gateway processes. Нужно **6 Paperclip agents**, использующих один adapter type `hermes_local`, но разные role instructions, budgets, context modes, toolsets и permissions.

Таблица 1 Hermes agents по отделениям

Отделение	Agent ID	Adapter	Toolsets
Div1 Executive	bos-div1-executive	hermes_local	safe, memory, messaging (no terminal)
Div2 Planning	bos-div2-planning	hermes_local	safe, web, search, file
Div3 Production	bos-div3-production	hermes_local	terminal, file, code_execution, delegation
Div4 Coordinator	bos-div4-coordinator	hermes_local	read/report only (fallback)
Div5 Knowledge	bos-div5-knowledge	hermes_local	safe, web, search, file, memory
Div6 Finance	bos-div6-finance	hermes_local	safe, file, memory (read-only)

Hermes Paperclip adapter запускает Hermes CLI в single-query mode, Hermes выполняет задачу и выходит. Session persistence работает через resume/session codec между heartbeats.

5. GSD-Pi через gsdpi_local

5.1 Custom adapter contract

GSD-Pi — локальный CLI agent для planning, implementing, verifying, tracking. Для Paperclip нужен **custom/external Paperclip adapter package** `gsdpi_local` — не BOS plugin worker, а standalone adapter, регистрируемый в Paperclip adapter registry через external adapter/plugin mechanism:

```
export const gsdpiLocal: ServerAdapterModule = {
  type: "gsdpi_local",

  async testEnvironment(ctx) {
    // check: gsd exists, cwd exists, Node >= required, .gsd usable
    return { status: "pass", checks: [] };
  },

  async execute(ctx: AdapterExecutionContext): Promise<AdapterExecutionResult> {
    // mode is adapter abstraction; translated to actual CLI per installed version
    const mode = ctx.config.mode ?? "headless-auto";
    const args = buildHeadlessArgs(ctx, mode); // e.g. ["headless","auto","--json"]

    const result = await runGsdPi({
      cwd: ctx.config.cwd,
      command: ctx.config.command ?? "gsd",
      args,
      context: buildGsdJobFromPaperclipContext(ctx),
      onStdout: chunk => ctx.onLog("stdout", chunk),
      onStderr: chunk => ctx.onLog("stderr", chunk),
      timeoutSec: Number(ctx.config.timeoutSec ?? 1800)
    });

    return {
      exitCode: result.exitCode,
      signal: result.signal ?? null,
      timedOut: result.timedOut,
      errorMessage: result.errorMessage ?? null,
      errorCode: result.errorCode ?? null,
      summary: result.summary,
      provider: result.provider ?? null,
      model: result.model ?? null,
      usage: result.usage ?? null,
      sessionParams: result.sessionParams ?? null,
      resultJson: {
        bos: {
          schemaVersion: "1.0",
          runId: ctx.runId,
          issueId: ctx.issueId,
          division: 4,
          role: "quality_ops",
          status: result.bosStatus,
          gateResults: result.gateResults,
          artifacts: result.artifacts,
          proposedPaperclipUpdates: result.proposedUpdates,
          artifactRefs: result.artifactRefs ?? [],
          riskSignals: result.riskSignals ?? [],
          nextRecommendedAgentId: result.nextRecommendedAgentId ?? null
        }
      }
    };
  }
};
```

5.2 Div4 quality gates

GSD-Pi headless mode (`gsd headless auto`, `gsd headless query`, `gsd headless recover`) — программная поверхность для automation. Adapter должен полагаться на structured output, exit codes, timeouts и artifacts, не на latency assumptions. `--json` / `--stream-json` где доступны; fail closed при отсутствии structured result.

Таблица 2 Div4 behavior через gsdpi_local

Input (Paperclip)	GSD-Pi action	Output (Paperclip)
Task ID, blueprint, acceptance criteria	Run verification + UAT	Gate verdict: pass/fail/partial
Changed files / branch / worktree	Regression + smoke checks	Evidence, logs, report artifact
Required eval gates, risk level	Inspect .gsd state	Proposed issue updates

5.3 Shared BosAdapterResult schema

Все BOS-aware adapters, включая `hermes_local` и `gsdpi_local`, возвращают `resultJson.bos` по единому контракту:

```
type BosAdapterResult = {
  schemaVersion: "1.0";
  runId: string;
  issueId: string;
  division: 1 | 2 | 3 | 4 | 5 | 6;
  role: string;
  status: "succeeded" | "failed" | "blocked" | "needs_input";
  summary?: string;
  gateResults?: Array<{
    gateId: string;
    verdict: "pass" | "fail" | "partial";
    evidenceArtifactIds: string[];
    summary: string;
  }>;
  artifacts?: Array<{
    kind: "blueprint" | "patch" | "report" | "test-log" | "decision-brief";
    path?: string;
    body?: string;
  }>;
  artifactRefs?: string[];
  proposedPaperclipUpdates?: Array<{
    field: string;
    value: unknown;
    reason: string;
  }>;
  riskSignals?: Array<{ signal: string; severity: "info" | "warn" | "critical" }>;
  nextRecommendedAgentId?: string | null;
};
```

6. BOS Light: что остается

BOS Light plugin — organizational/governance layer только. Execution lifecycle (heartbeats, sessions, logs, costs, workspaces, budgets) остается в Paperclip.

Таблица 3 Что делает BOS Light plugin

Функция	Как	Не делает
Division metadata	Overlay: producer_division, bos_status	Не spawn agents
BPI scoring	piko:bpi-score agent tool	Не invoke adapterType directly
Betting Table	Dashboard widget, native approval	Не approve сам
Blueprints	piko:blueprint-gen, issue document	Не execute
Eval Gates	Review checklist, gate results	Не replace Paperclip reviews
Circuit Breaker	Polling fallback, escalation	Не modify issue status directly
Routing	Assign agent_id by division/rules	Не manage adapter lifecycle
Result governance	Parse resultJson.bos, apply allowed updates	Не bypass Paperclip governance

7. Обновленный план фаз

Таблица 4 Изменения от v1.2 к v1.3

Фаза	v1.2 (было)	v1.3 (исправлено)
Phase 1	7 native agents (claude/codex)	7 agents через Paperclip adapters (hermes_local + fallback)
Phase 2	BPI + Betting Table + Blueprints	Без изменений
Phase 3	Circuit Breaker + Eval Gates	Без изменений
Phase 4	Div7 Decision Protocol	Без изменений
Phase 5	Hermes adapter для Div3 only	Hermes profiles/agents для Div1-3,5-6 через hermes_local
Phase 6	GSD-Pi local-only optional	gsdpi_local adapter для Div4 (custom Paperclip adapter)

8. Финальная схема

BOS Light v1.3 — Implementation Baseline

```
Paperclip Company
  ├─ BOS Div1 Executive        adapterType: hermes_local
  ├─ BOS Div2 Planning         adapterType: hermes_local
  ├─ BOS Div3 Production       adapterType: hermes_local
  ├─ BOS Div4 Quality/Ops      adapterType: gsdpi_local
  ├─ BOS Div4 Coordinator      adapterType: hermes_local (fallback)
  ├─ BOS Div5 Knowledge/Sec    adapterType: hermes_local
  └─ BOS Div6 Finance/Res      adapterType: hermes_local

BOS Light Plugin
  ├─ adds BOS metadata (bpi_score, bos_status, division)
  ├─ routes tasks by division → agent assignment
  ├─ enforces BPI / Betting Table / Eval Gates
  ├─ parses resultJson.bos from adapters
  └─ applies allowed state transitions
```

Короткая формула: **Hermes/GSD-Pi — не external executors, а Paperclip agents с adapterType. BOS Light — organizational intelligence, не orchestration platform.**

9. Adapter Hardening Checklists

9.1 Hermes adapter production gates

hermes_local — preflight checklist

- Pin `hermes-paperclip-adapter` version (no floating)
- Run Paperclip heartbeat smoke test per role (Div1-Div6)
- Verify `ctx.context` vs `ctx.config` task-context mapping
- Verify latest user comment injection into prompt
- Verify session resume does not persist malformed IDs
- Verify AGENTS.md/SOUL.md role context is injected or explicitly embedded in `promptTemplate`
- Verify no direct Paperclip API curl hacks required from inside Hermes
- Verify done/completed runs do not re-wake into duplicate execution
- Verify toolset restrictions per profile (terminal only for Div3, no terminal for Div1)

9.2 GSD-Pi adapter production gates

gsdpi_local — preflight checklist

- Require `@opengsd/gsd-pi` (npm scoped), not old unscoped `gsd-pi`
- Run `gsd --version` and `gsd headless --help` during `testEnvironment()`
- Verify Node.js >= 22
- Verify repo root and `.gsd` state exists and is writable
- Verify clean/acceptable git state before source-writing gates
- Verify headless output mode actually supported in installed version
- Fail closed on dirty worktree, missing .gsd, invalid worktree, or missing structured result
- Never rely on markdown projections as sole source of truth if DB is available
- Attach raw logs as artifact on failed/blocked run
- Single-host only: never share `.gsd/gsd.db*` across NFS/network

10. Routing Chain

Правильная цепочка: BOS → agent_id → Paperclip → adapter

```
BOS Light routing policy
  → chooses assignee agent_id (by division, rules, BPI)
  → Paperclip agent assignment
  → Paperclip heartbeat invocation
  → adapterType resolution (hermes_local / gsdpi_local / claude_local)
  → adapter execute()
  → runtime (Hermes / GSD-Pi / Claude)
  → resultJson.bos
  → BOS Light result governance
```

BOS Light **не вызывает adapterType напрямую** и **не управляет adapter lifecycle**. BOS выбирает agent_id; Paperclip выбирает adapter через agent config.